Scaling Laws and "Large" Language Models

Meng Jiang[1]

[1]Department of Computer Science and Engineering (CSE)
University of Notre Dame

CSE 60556 LLM

Table of Contents

Table of Contents

What are scaling laws?

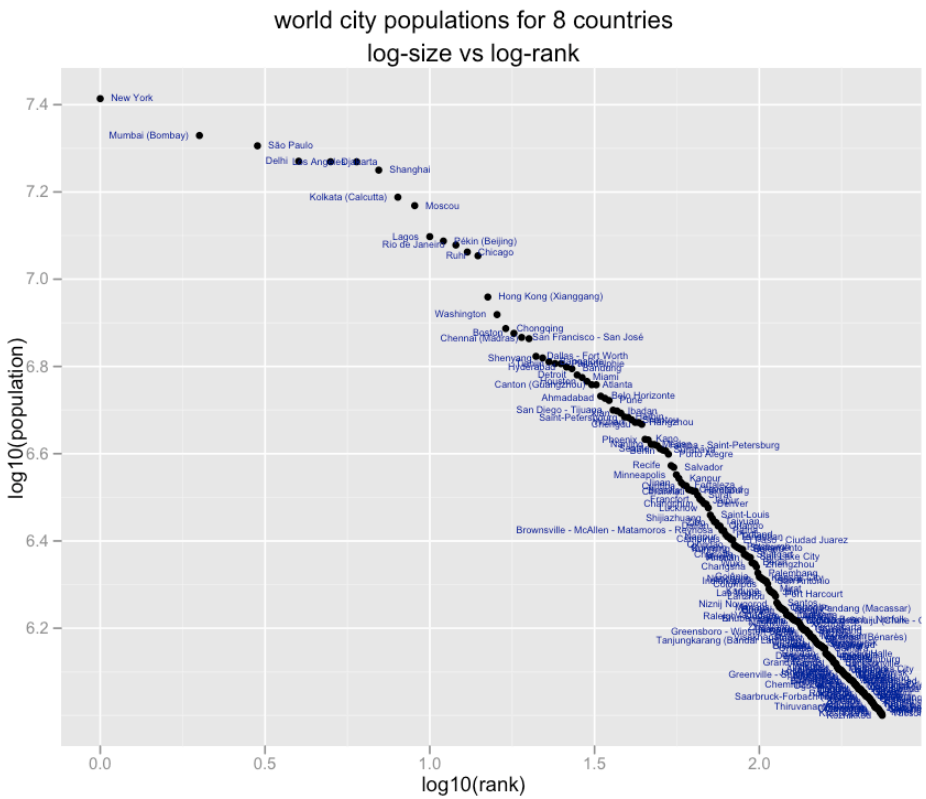
Scaling laws are one of the most critical empirical findings in deep learning.

N	**Model** size, measured in parameter count
D	Training **dataset** size, usually measured in token count
C	Training **compute** in Floating Point Operations per sec. (FLOPs)
L	Test loss, also refer to training **loss** (they are strongly correlated)

The training loss L decreases predictably as we scale up model size N, dataset size D, and compute C, following a power-law curve, which appears as a straight line on a log-log plot.

Credit to: *Scaling Law, Carefully*; Lil'Log
https://lilianweng.github.io/posts/2026-06-24-scaling-laws/

Power laws are everywhere



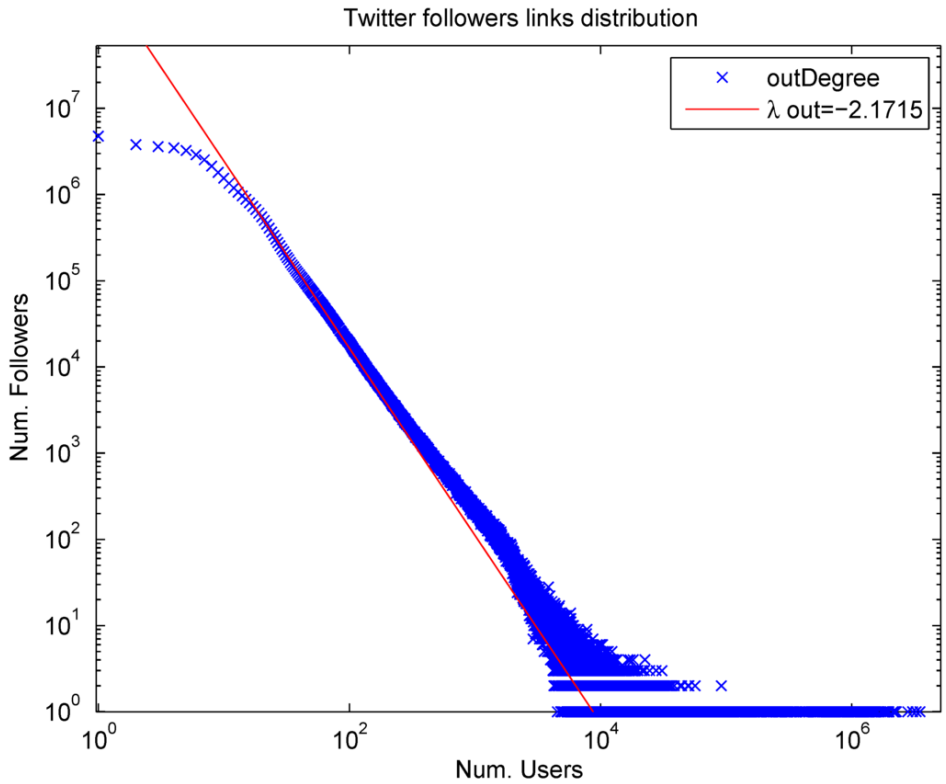


Image credit to https://brenocon.com/blog/wp-content/uploads/2009/05/picture-4.png

Why scaling laws are important for deep learning?

This predictability makes scaling laws highly valuable in practice.

A common workflow is to fit scaling laws on a handful of small runs and then extrapolate to estimate the token and compute requirements for larger models, e.g.,

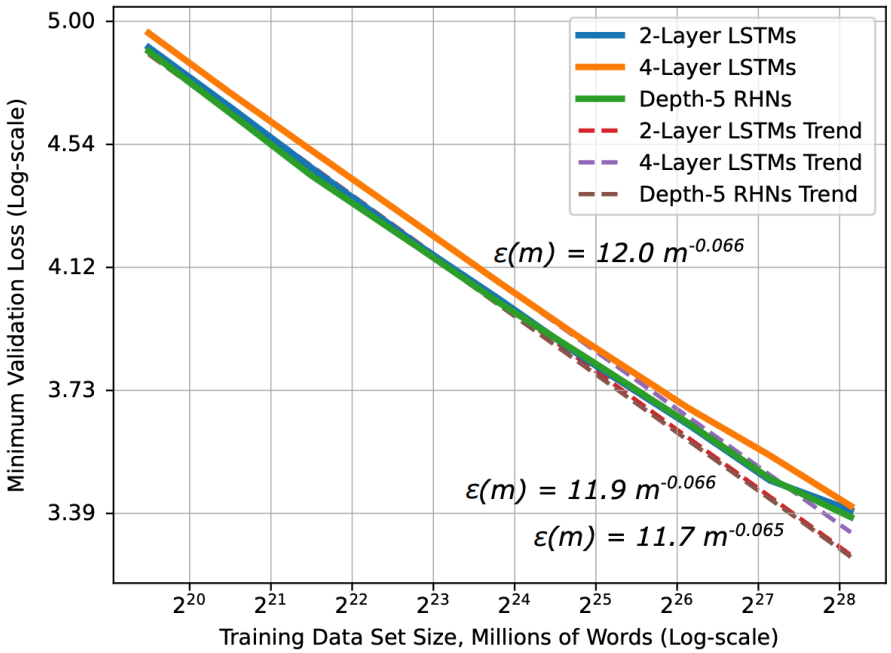
from Kaplan et al. 2020 (Scaling Laws for Neural Language Models):

$$L(N, D) = \left[\left(\frac{a}{N} \right)^{\frac{\alpha}{\beta}} + \frac{b}{D} \right]^{\beta}, \tag{1}$$

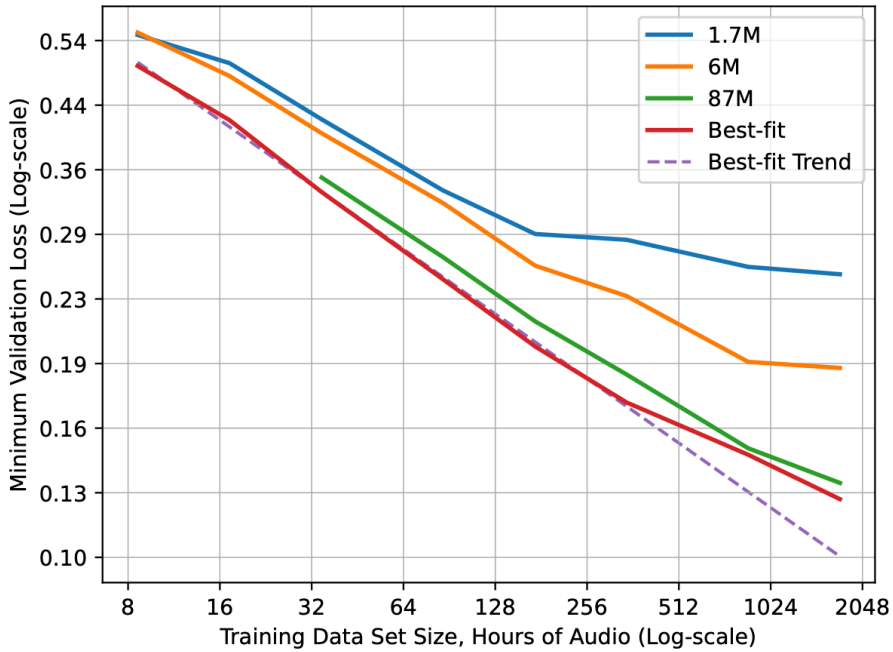
$$C \approx 6 \times N \times D. \tag{2}$$

Scaling Laws in Deep Learning

Language modeling:



Speech modeling:



The losses of small models plateau when training data becomes large.

from Hestness et al. 2017: Deep Learning Scaling is Predictable, Empirically

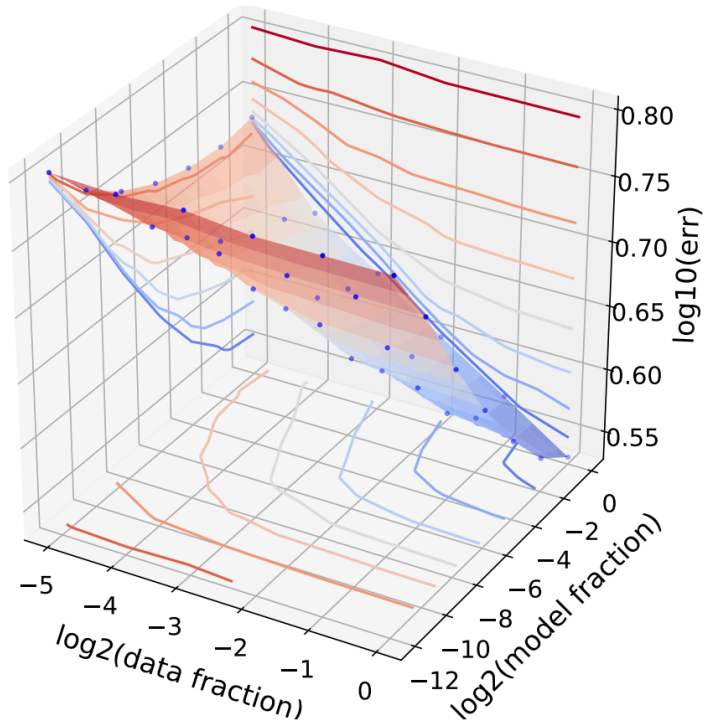
Scaling Laws: 3D Landscape

Modeling generalization error $\hat{L}$ as a joint function of both model size N and data size D, across a diverse set of architectures (ResNet, LSTM, Transformer) and optimizers (Adam, SGD variants):

$$\hat{L}(D, N) \approx \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} + E, \qquad (3)$$

where $A > 0$, $B > 0$, $\alpha \geq 0$, $\beta \geq 0$ are scalar constants and E is not dependent on either N or D.

Wiki103 error landscape:

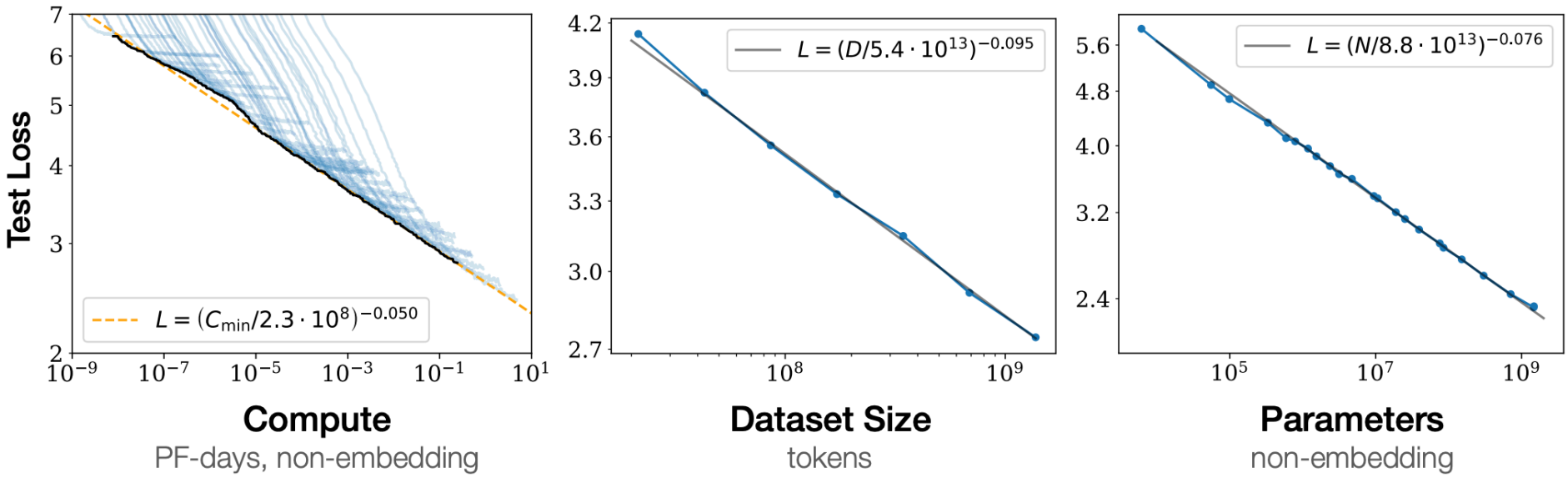


from Rosenfeld et al. 2019: A Constructive Prediction of the Generalization Error Across Scales

Table of Contents

Kaplan et al.'s Scaling Laws

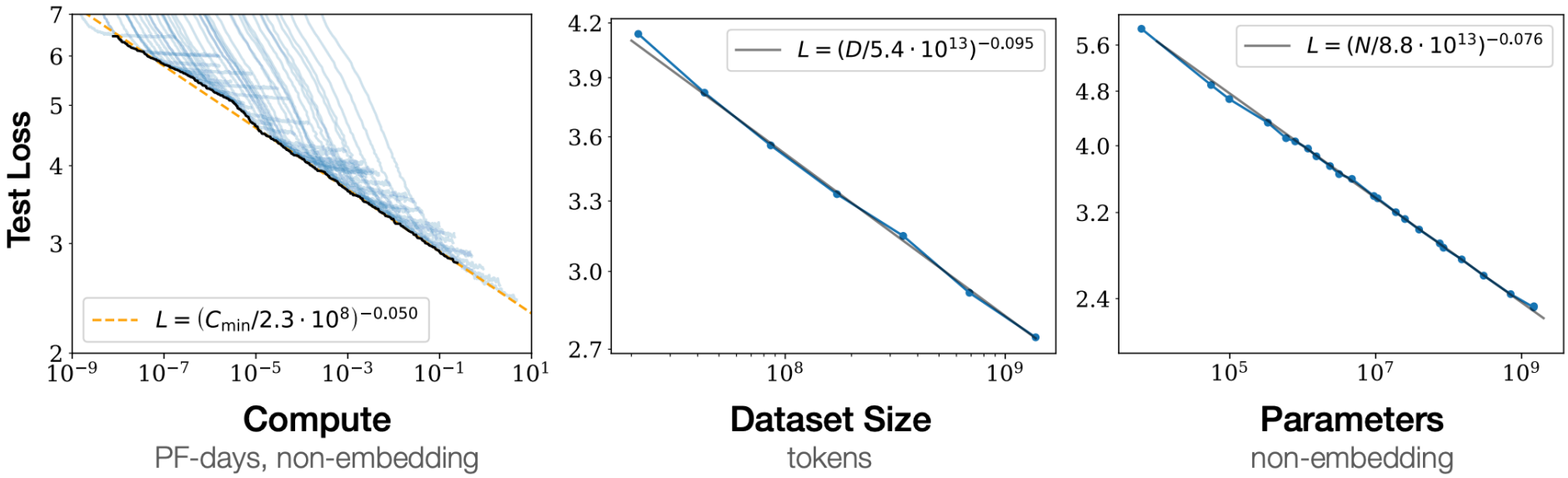
The loss L scales as a power law with N, D, and C individually:



with model size ranging from 768M to 1.5B non-embedding parameters and dataset size from 22M to 23B tokens. All training used a learning rate schedule with a 3,000 step linear warmup, followed by a decay to zero.

Kaplan et al.'s Scaling Laws

The loss L scales as a power law with N, D, and C individually:



with model size ranging from 768M to 1.5B non-embedding parameters and dataset size from 22M to 23B tokens. All training used a learning rate schedule with a 3,000 step linear warmup, followed by a decay to zero.

Kaplan et al.'s Scaling Laws

Observation: For a $10x$ increase in compute C, scale the model size N by $5.5x$ but the training tokens by only $1.8x$. And $C \propto N \times D$.

Suggestion: Given a fixed compute budget, it is more efficient to train a very large model and stop before convergence than to train a smaller model all the way to convergence.

Chinchilla scaling laws disagree: Kaplan et al. overestimated the optimal model size as their fitted exponent was larger, leaving large models badly **undertrained**.

from Hoffmann et al. 2022: Training Compute-Optimal Large Language Models

Prove $C \approx 6ND$

Let's approximate the number of training FLOPs needed based on N and D. Each multiply-add is counted as ~ 2 FLOPs.

Operations	**Parameters**	**FLOPs per Token**
Embed	$(n_{vocab} + n_{ctx})d_{model}$	$4d_{model}$
Attention: QKV	$n_{layer}d_{model} \cdot 3d_{attn}$	$2n_{layer}d_{model} \cdot 3d_{attn}$
Attention: Mask	–	$2n_{layer}n_{ctx}d_{attn}$
Attention: Project	$n_{layer}d_{attn}d_{model}$	$2n_{layer}d_{attn}d_{embd}$
Feedforward	$n_{layer} \cdot 2d_{model}d_{ff}$	$2n_{layer} \cdot 2d_{model}d_{ff}$
De-embed	–	$2d_{model}n_{vocab}$
Total (non-embedding)	$N = 2d_{model}n_{layer}(2d_{attn} + d_{ff})$	$C_{forward} = 2N + 2n_{layer}n_{ctx}d_{attn}$

We count backward-pass FLOPs as **twice** as the forward-pass FLOPs, because backpropagation runs two matrix multiplications, for gradients w.r.t. the input activations and weights. The training FLOPs per token are approximately $6N$. The total FLOPs for training over D tokens are $C \approx 6ND$.

Chinchilla Scaling Laws: Allocate resources given $C \approx 6ND$

When are have only limited FLOPs (a given number of GPUs running for a given period of time), how should we choose between more data tokens and more model parameters?

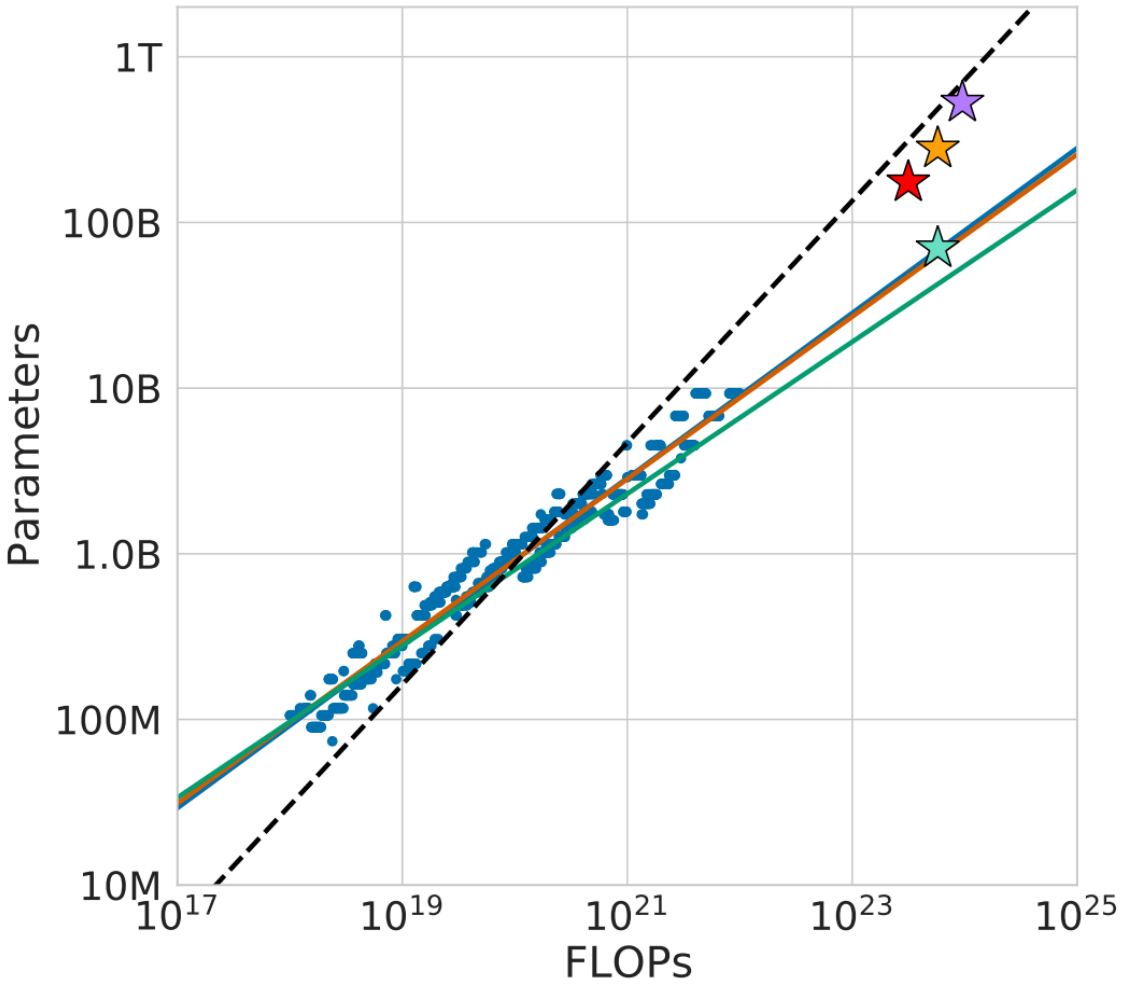
$$N_{opt}(C), D_{opt}(C) = \arg\min_{s.t. FLOPs(N,D)=C} \hat{L}(N, D). \tag{4}$$

The experiments scanned over 400 models, with sizes from 70M to over 16B parameters (including embedding parameters) and training tokens from 5B to 500B.

The experiments were under the assumption that every training tokekn is unique. All runs used a cosine learning-rate schedule decaying by $10x$ over the training horizon.

from Hoffmann et al. 2022: Training Compute-Optimal Large Language Models

Chinchilla Scaling Laws: Learning from Statistics



Approach 1
Approach 2
Approach 3
Kaplan et al (2020)

Chinchilla (70B)
Gopher (280B)
GPT-3 (175B)
Megatron-Turing NLG (530B)

Chinchilla Scaling Laws: Design

Under the same compute budget as Gopher, Chinchilla was $4x$ smaller but trained on roughly $4x$ more tokens and it outperformed Gopher:

Model	**Size (# Parameters)**	**Training Tokens**
LaMDA (Thoppilan et al. 2022)	137 billion	168 billion
GPT-3 (Brown et al. 2020)	175 billion	300 billion
Jurassic (Lieber et al. 2021)	178 billion	300 billion
Gopher (Rae et al. 2021)	280 billion	300 billion
MT-NLG 530B (Smith et al. 2022)	530 billion	270 billion
Chinchilla (Hoffmann et al. 2022)	70 billion	1.4 trillion

Reconciling Kaplan and Chinchilla

The Chinchilla scaling laws disagree with Kaplan et al. as follows:

- Instead of "grow the model faster than the data" ($N_{opt} \propto C^{0.73}$), for every doubling of model size, you should also double the number of training tokens ($N_{opt} \propto C^{0.5}$).
- Instead of "train a big model and stop before convergence," you should train a smaller model on more data.

Why do they disagree so much?

- Kaplan et al. experimented mostly on small models.
- Embedding parameters count matters for small models.

Table of Contents

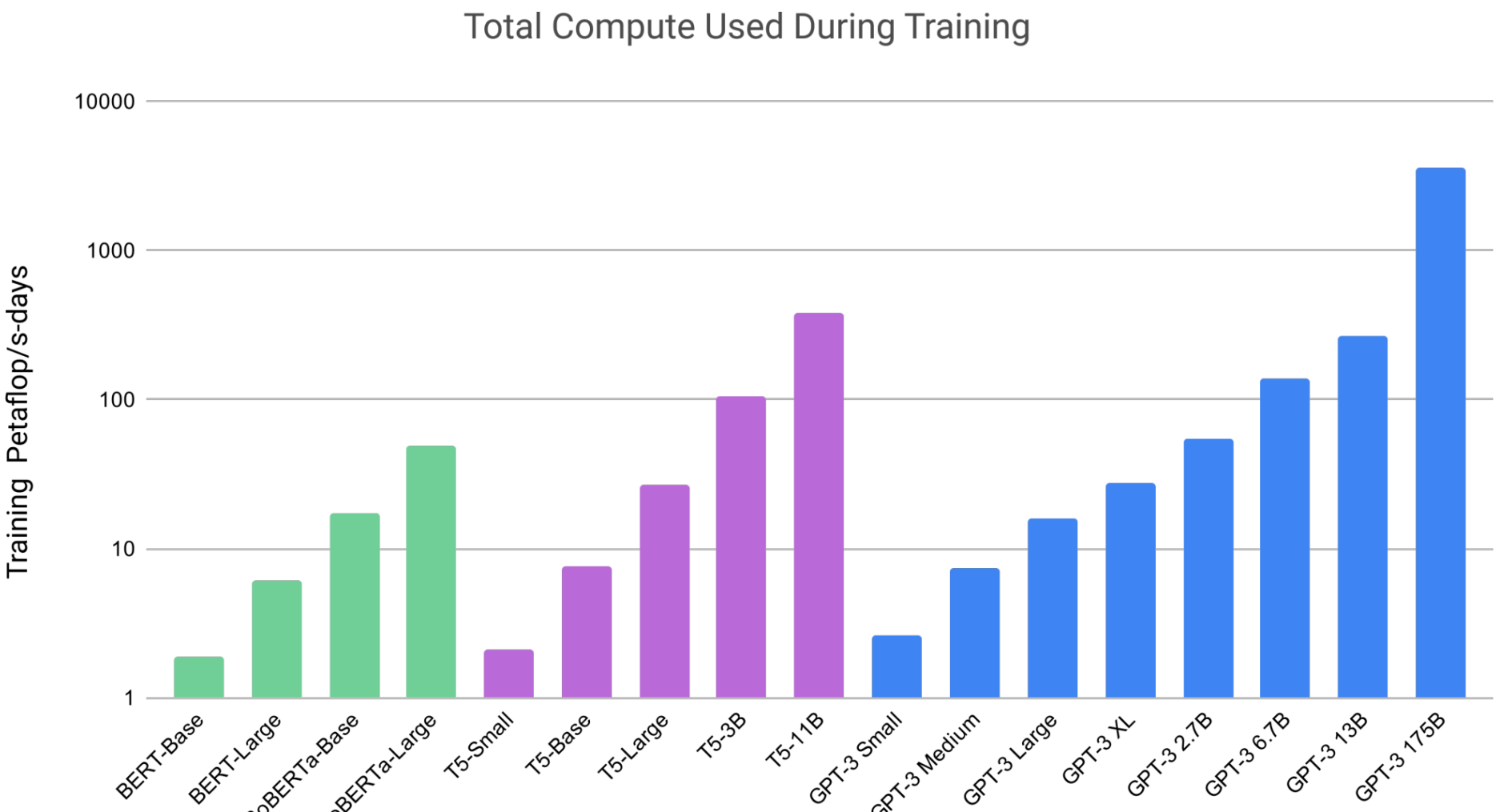
Applying Scaling Laws: Model Size and Datasets

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

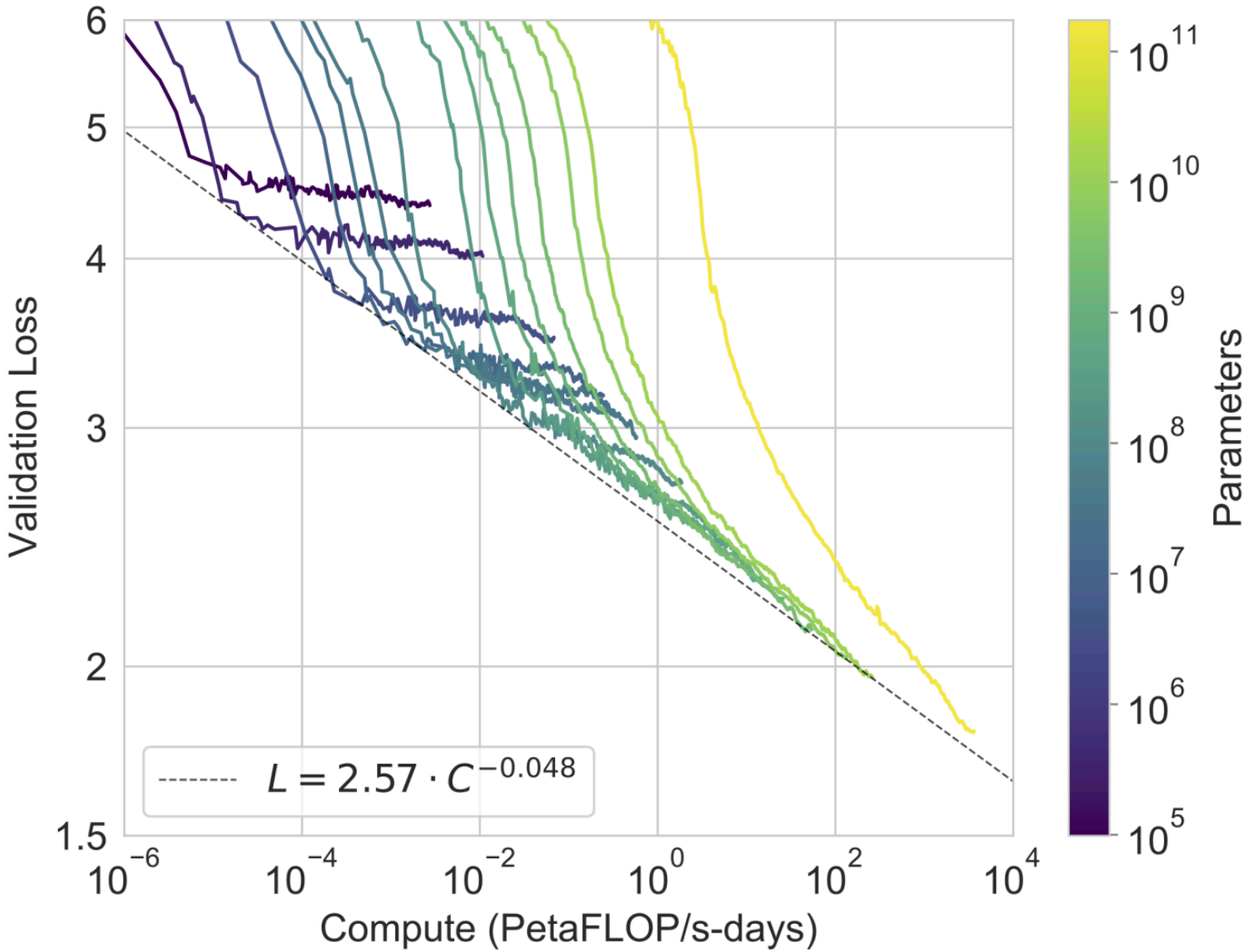
Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

from Brown et al. 2020: Language Models are Few-Shot Learners

Applying Scaling Laws: Crazy FLOPs



Smooth Scaling on Language Modeling

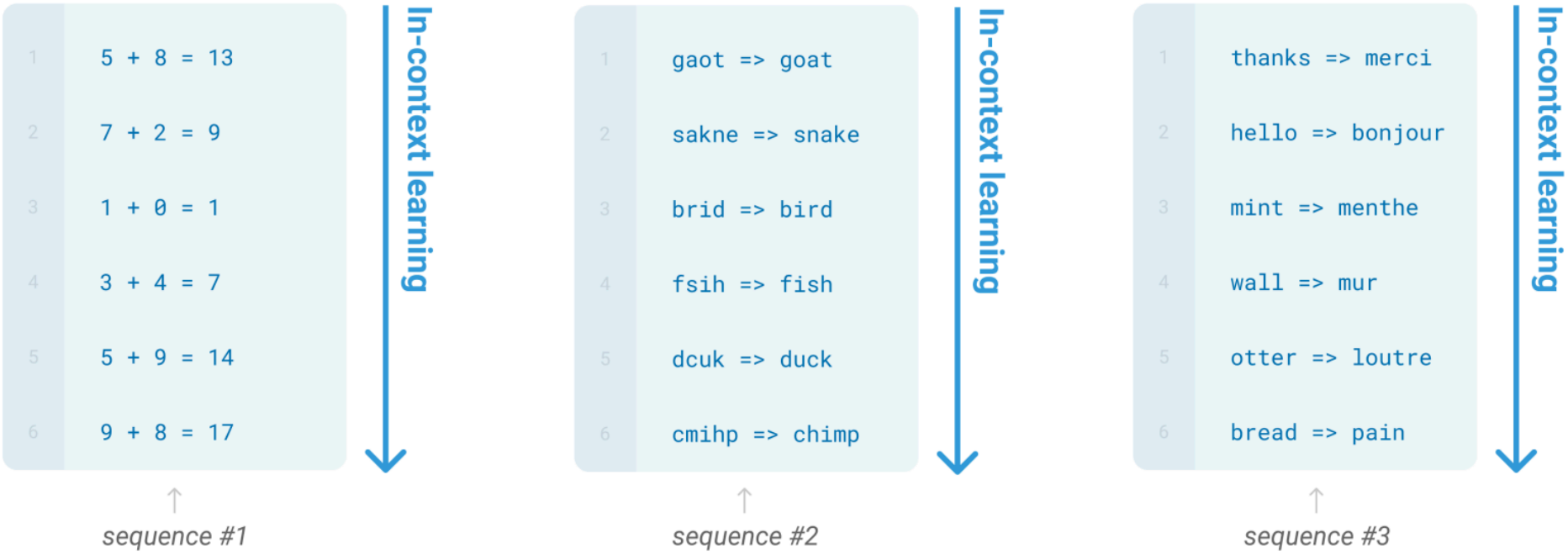


In-Context Learning (ICL)

ICL: The ability of LMs to learn tasks directly from examples provided in the input prompt, without parameter updates.

Features:

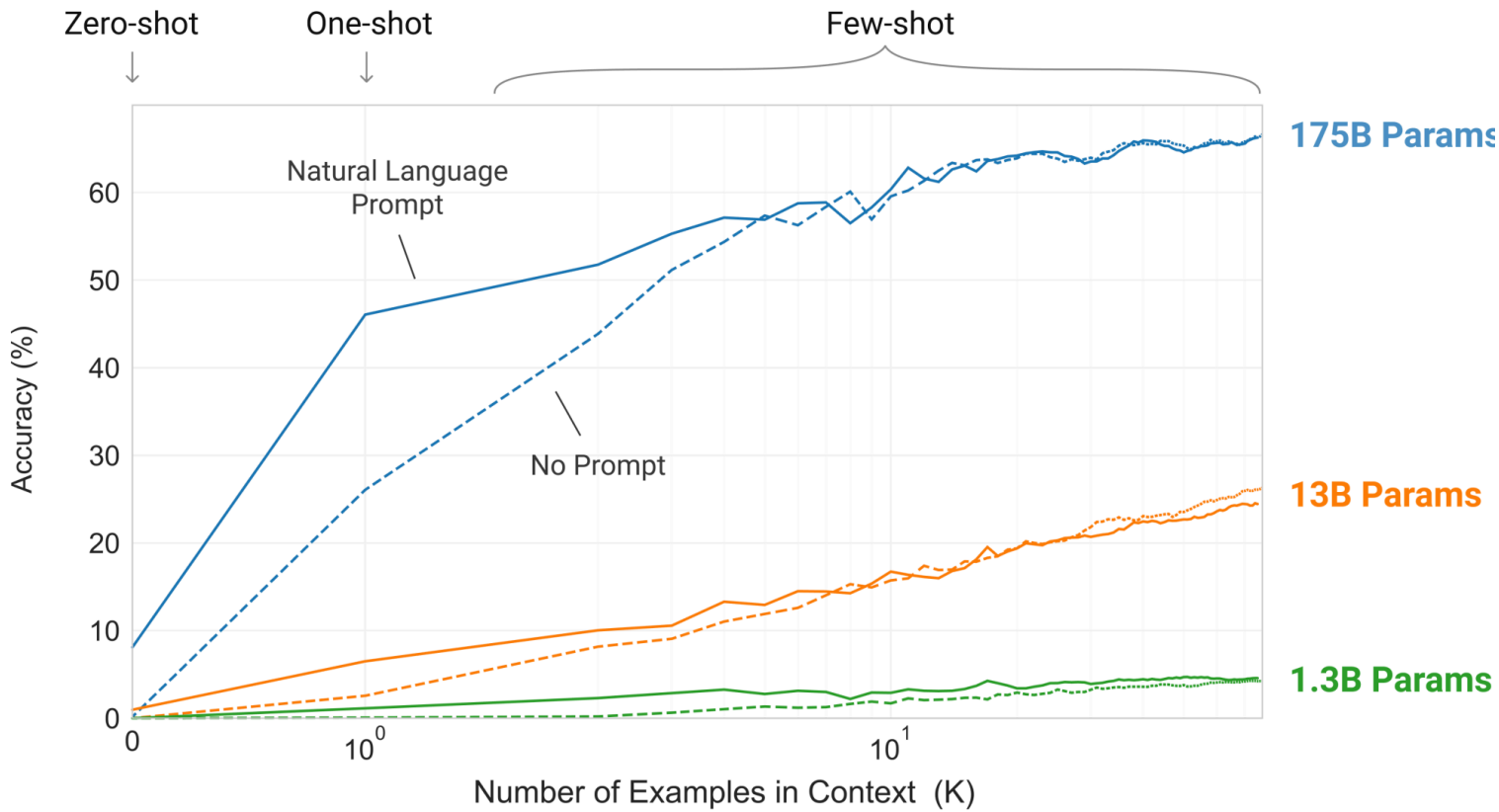
- Emerges in large scale models only
- Works for many tasks (translation, math, classification, etc.)



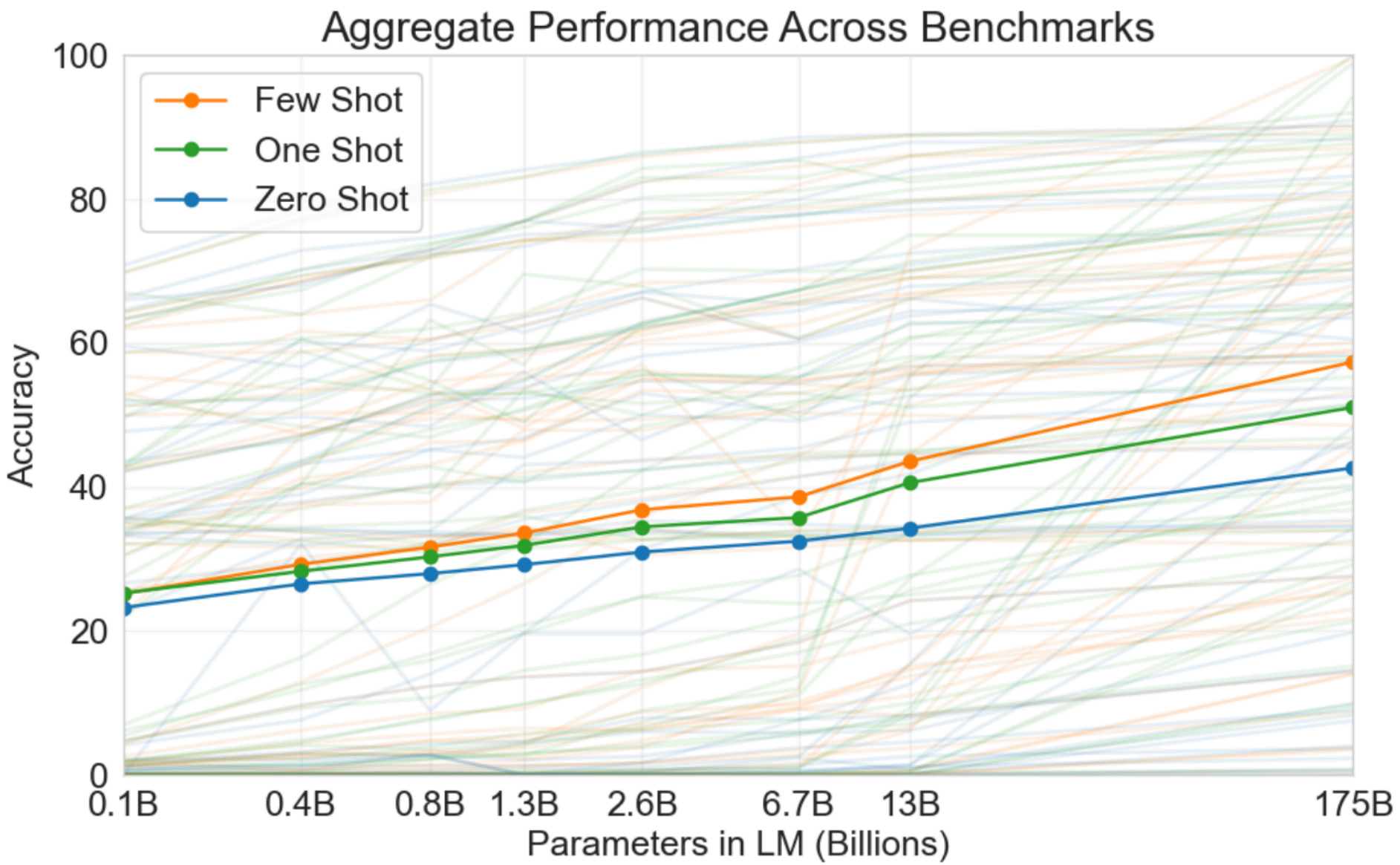
In-Context Learning (ICL)

- Zero-shot: model relies on instructions alone, no instructions provided
- One-shot: model learns from one example
- Few-shot: model learns from multiple examples

Performance on removing random symbols from a word:



ICL emerges in large models, aggregated across 42 tasks



On Open-Domain Question Answering: TriviaQA

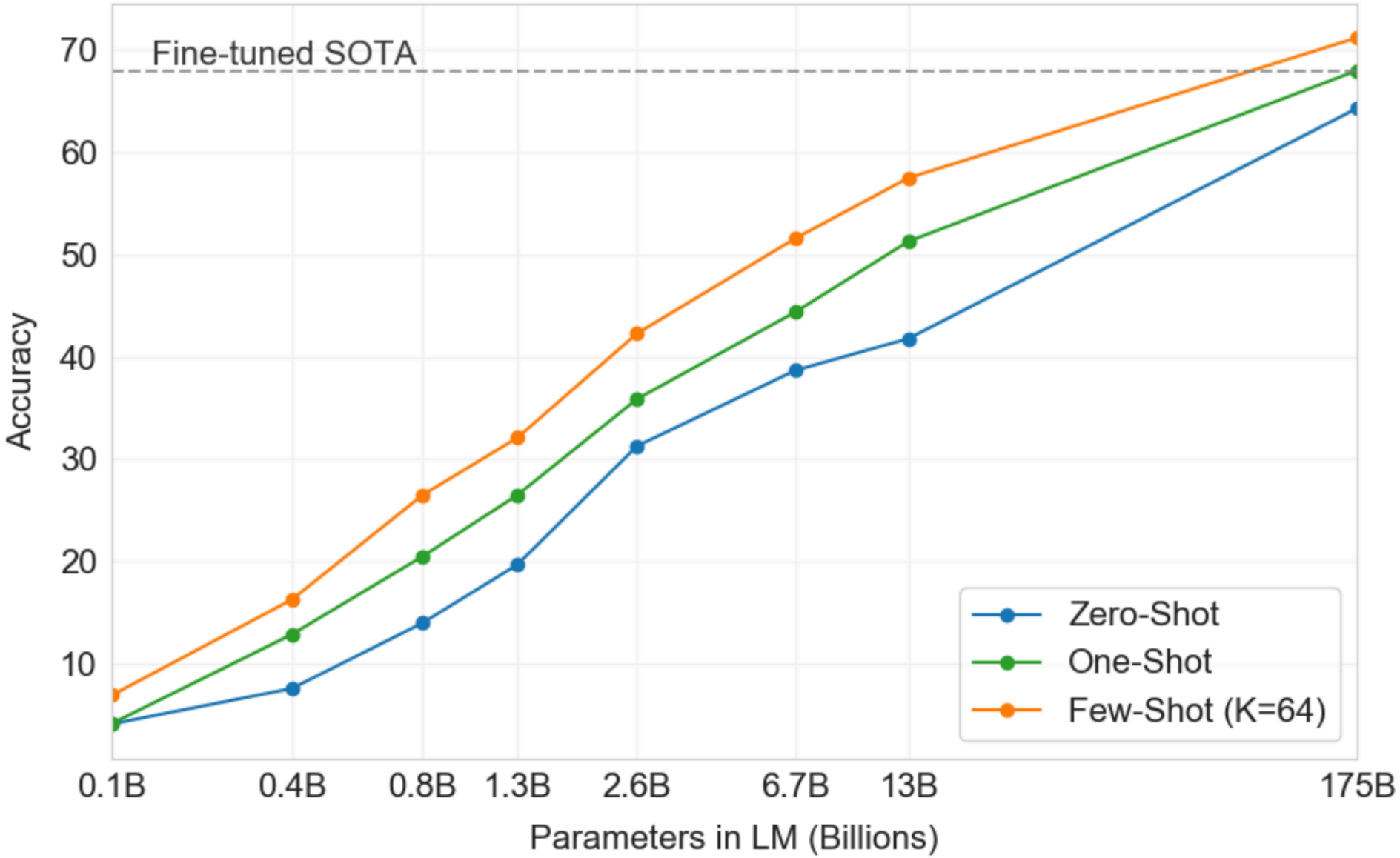


Table of Contents

Why ICL works: Meta-Learning Hypothesis

- ICL works because LLMs behave like meta-learners - they implicitly perform fast adaptation by simulating gradient descent updates within their forward pass.
- Attention layers can internally re-weight representations of tokens in the hidden states (the activations that flow through the network). This redistribution looks similar to doing a few steps of gradient descent on demo examples.

from Brown et al. 2020: Language Models are Few-Shot Learners

Why ICL works: Bayesian Inference View

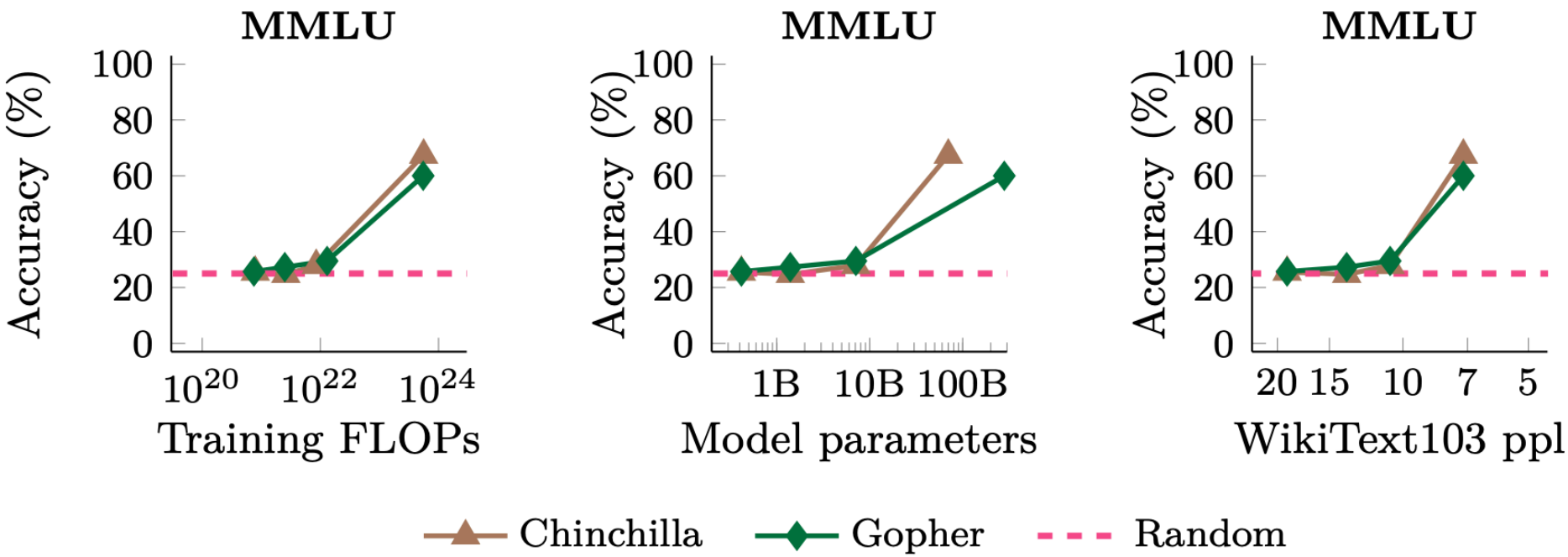
- The prompt examples act as evidence about which task is being performed.
- The model narrows down its "hypothesis space" using the prior knowledge (inferred from prompt examples).
- Output is a posterior guess: the most probable continuation given the observed evidence.

$$p(\text{output}|\text{prompt}) = \int_{\text{concept}} p(\text{output}|\text{concept}, \text{prompt}) \cdot p(\text{concept}|\text{prompt}) \cdot d(\text{concept}) \quad (5)$$

from Xie et al. 2021: An Explanation of In-Context Learning as Implicit Bayesian Inference

Emergent Abilities of LARGE Language Models

Emergent abilities were not present in smaller models but are present in larger models. Thus, they cannot be predicted simply by extrapolating the performance of smaller models.



from Wei et al. 2022: Emergent Abilities of Large Language Models

Massive Multitask Language Understanding (MMLU)

- The benchmark covers 57 subjects across STEM, the humanities, the social sciences, and more. It ranges in difficulty from an elementary level to an advanced professional level, and it tests both world knowledge and problem solving ability.
- Few-shot models up to 13B parameters achieve random chance performance of 25% accuracy, but the 175B parameter GPT-3 model reaches a much higher 43.9% accuracy.
- Unlike human professionals GPT-3 does not excel at any single subject. Instead, we find that performance is lopsided, with GPT-3 having almost 70% accuracy for its best subject but near-random performance for several other subjects.

from Hendrycks et al. 2021: Measuring Massive Multitask Language Understanding

Emergent Abilities: More Results

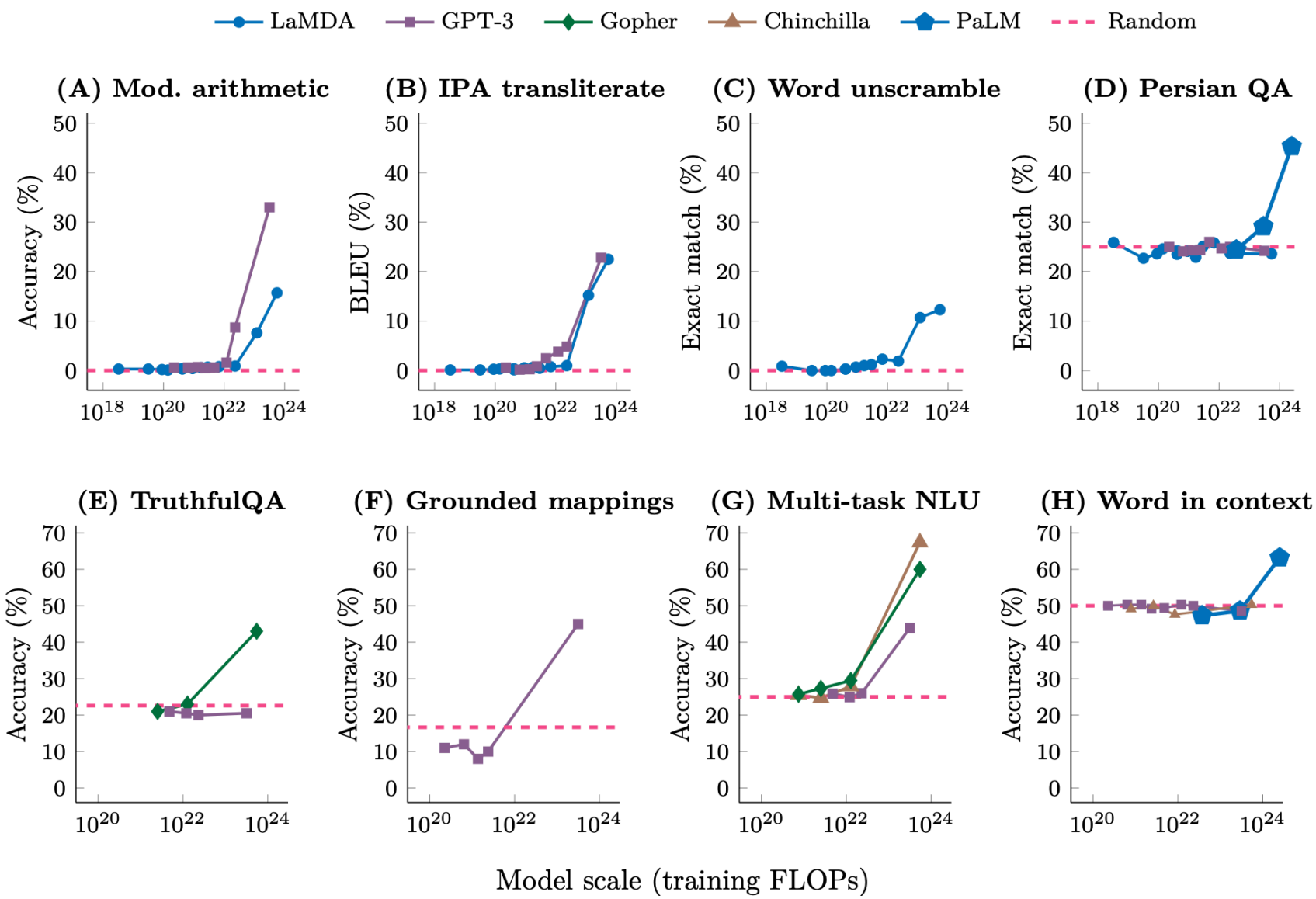


Table of Contents

Why MoE?

Challenge: Model strength increases with more parameters, but this also leads to an increase in compute.

Goal: To increase parameters without increasing compute.

Idea: Replace the feed-forward neural (FFN) layers with many FFNs and a selector layer (or "routing algorithm").

from Jiang et al. 2024: Mixtral of Experts (known as Mixtral 8x7B)

from Dai et al. 2024: DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

MoE: Implementation

- Routing algorithms: Decides which experts are activated for a given input (top-k routing, learned gates, fixed routing, RL-based routing);
- Load balancing: Must ensure that each expert is used evenly to avoid imbalance (auxiliary losses, noise injection, capacity constraints);
- Total parameters: All the weights stored in the model;
- Active parameters: The subset of those weights that are actually used in computing a single forward pass. In dense models, active parameters = total parameters;
- Higher expert count leads to higher total parameter count, but also harder routing and balancing;
- Larger expert size leads to more representational power, but uses more memory/compute;
- Frequency: Determines where in the model MoEs are inserted (every layer, every other layer, etc.)

Mixtral Model Architecture

Suppose the MoE model has n expert networks $\{E_0, \ldots, E_i, \ldots, E_{n-1}\}$.

Given input x,

- $G(x)_i$ is the n-dim. output of the gating network for the i-th expert:

$$G(x) := \mathrm{softmax}(\mathrm{topK}(x \cdot W_g)); \tag{6}$$

- $E_i(x)$ is the output of the i-th expert network.

The output is the weighted sum of the outputs of the expert networks:

$$\sum_{i=0}^{n-1} G(x)_i \cdot E_i(x). \tag{7}$$

If the gating vector is **sparse**, we can avoid computing the outputs of experts whose gates are zero.

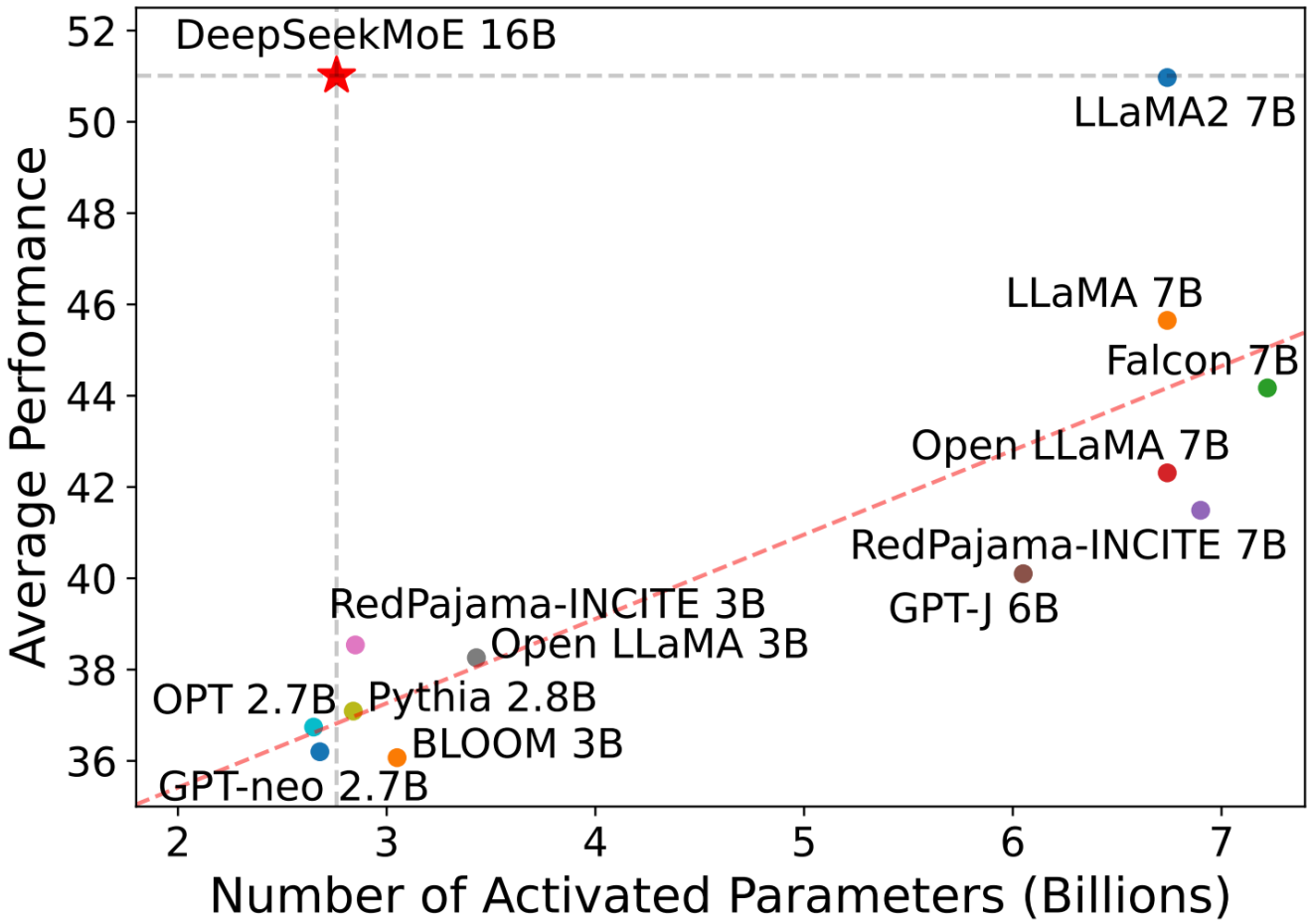
Mixtral Results

Mixtral outperforms or matches Llama-2 70B performance on popular benchmarks while using 5x fewer active parameters during inference.

Model	Active Params	MMLU	HellaS	WinoG	PIQA	Arc-e	Arc-c	NQ	TriQA	HumanE	MBPP	Math	GSM8K
LLaMA 2 7B	7B	44.4%	77.1%	69.5%	77.9%	68.7%	43.2%	17.5%	56.6%	11.6%	26.1%	3.9%	16.0%
LLaMA 2 13B	13B	55.6%	80.7%	72.9%	80.8%	75.2%	48.8%	16.7%	64.0%	18.9%	35.4%	6.0%	34.3%
LLaMA 1 33B	33B	56.8%	83.7%	76.2%	82.2%	79.6%	54.4%	24.1%	68.5%	25.0%	40.9%	8.4%	44.1%
LLaMA 2 70B	70B	69.9%	**85.4%**	**80.4%**	82.6%	79.9%	56.5%	25.4%	**73.0%**	29.3%	49.8%	13.8%	69.6%
Mistral 7B	7B	62.5%	81.0%	74.2%	82.2%	80.5%	54.9%	23.2%	62.5%	26.2%	50.2%	12.7%	50.0%
Mixtral 8x7B	13B	**70.6%**	84.4%	77.2%	**83.6%**	**83.1%**	**59.7%**	**30.6%**	71.5%	**40.2%**	**60.7%**	**28.4%**	**74.4%**

from Jiang et al. 2024: Mixtral of Experts (known as Mixtral 8x7B)

DeepSeekMoE Results



from Dai et al. 2024: DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

Challenges in MoE

- **Knowledge Hybridity:** Existing MoE practices often employ a limited number of experts (e.g., 8 or 16), and thus tokens assigned to a specific expert will be likely to cover diverse knowledge. Consequently, the designated expert will intend to assemble vastly different types of knowledge in its parameters, which are hard to utilize simultaneously.
- **Knowledge Redundancy:** Tokens assigned to different experts may require common knowledge. As a result, multiple experts may converge in acquiring shared knowledge in their respective parameters, thereby leading to redundancy in expert parameters. These issues collectively hinder the expert specialization in existing MoE practices, preventing them from reaching the theoretical upper-bound performance of MoE models.

DeepSeekMoE Innovations

- **Fine-Grained Expert Segmentation:** While maintaining the number of parameters constant, segment the experts into a finer grain by splitting the FFN intermediate hidden dimension. It allows diverse knowledge to be decomposed more finely and be learned more precisely into different experts, where each expert will retain a higher level of specialization. The flexibility in combining activated experts contributes to a more accurate and targeted knowledge acquisition.
- **Shared Expert Isolation:** Isolate certain experts to serve as shared experts that are always activated, aiming at capturing and consolidating common knowledge across varying contexts. Through compressing common knowledge into these shared experts, redundancy among other routed experts will be mitigated. This can enhance the parameter efficiency and ensure that each routed expert retains specialized by focusing on distinctive aspects.

Takeaways

- Scaling laws exist in deep learning.
- Scaling laws are found in language models: Kaplan et al. vs Chinchilla. It's about Compute, Data, and Model size.
- GPT-3 was $10x$ larger than GPT-2. In-context learning abilities emerged: zero-shot, one-shot, and few-shot generalization.
- Mixture-of-Experts architecture has been adopted in many advanced models: Mixtral, DeepSeekMoE, Gemma4, etc.